1. Pandas Data Structures

In Pandas, there are two main data structures:1. **Series and 2. DataFrame**

A **Series** in pandas is a **one-dimensional labeled array** capable of holding any data type (integers, floats, strings, Python objects).

Think of it as:

- A **single column of a table**
- Or a **labeled list**

🔷 1. Key Features of Series

- ✅ One-dimensional (1D)
- ✅ Has **values + index (labels)**
- ✅ Can store **heterogeneous data**
- ✅ Supports **vectorized operations**
- ✅ Automatically aligns data by index

✅ (A) Series

👉 **1D labeled array**

(a) From a List

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
```

✔ Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

👉 Default index: 0, 1, 2, 3

b) With Custom Index

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s)
```

✔ Output:

```
a    10
b    20
```

```
c    30
dtype: int64
```

(c) From Dictionary

```
data = {'a': 100, 'b': 200, 'c': 300}
s = pd.Series(data)
print(s)
```

✔ Output:

```
a    100
b    200
c    300
```

Accessing Elements

```
print(s['a'])   # By label
print(s[0])     # By position
```

🔷 Slicing

```
s = pd.Series([10, 20, 30, 40, 50])

print(s[1:4])
```

✔ Output:

```
1    20
2    30
3    40
```

🔷 Operations on Series

✅ Arithmetic Operations

```
s = pd.Series([1, 2, 3])

print(s + 5)
```

✔ Output:

```
0    6
1    7
2    8
```

✅ Vectorized Operations

```
print(s * 2)
```

🔷 Important Attributes

```
print(s.index)   # Index labels
print(s.values)  # Data values
```

```
print(s.dtype)    # Data type
print(s.shape)    # Shape
```

Handling Missing Values

```
import numpy as np

s = pd.Series([1, 2, np.nan, 4])

print(s.isnull())
print(s.dropna())
```

0 False
1 False
2 True
3 False
dtype: bool
0 1.0
1 2.0
3 4.0
dtype: float64

🔷 Series Alignment

```
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([4, 5, 6], index=['b', 'c', 'd'])

print(s1 + s2)
```

✔ Output:

```
a    NaN
b    6
c    8
d    NaN
```

👉 Data aligns based on index labels

🔷 Real-Life Example

```
marks = pd.Series([85, 90, 78], index=['Math', 'Science', 'English'])

print("Marks in Science:", marks['Science'])
```

✅ DataFrame

Pandas DataFrame Data Structure

A **DataFrame** in pandas is a **two-dimensional, labeled data structure** with rows and columns. It is the most commonly used structure for handling real-world datasets.
👉 Think of it as:

- An **Excel sheet**
- A **database table**
- A collection of multiple **Series**

🔷 1. Key Features of DataFrame

- ✅ Two-dimensional (rows + columns)
- ✅ Each column can have **different data types**

- ✅ Has **row index + column labels**
- ✅ Supports **vectorized operations**
- ✅ Handles **missing values efficiently**

👉 **2D labeled data structure (rows + columns)**

🔹 Creation of DataFrame

✅ From Dictionary

```
data = {
    "Name": ["A", "B", "C"],
    "Marks": [85, 90, 78]
}

df = pd.DataFrame(data)
print(df)
```

✔ Output:

```
  Name  Marks
0    A     85
1    B     90
2    C     78
```

✅ From List of Lists

data = [['Amit', 20], ['Riya', 21], ['John', 22]]

df = pd.DataFrame(data, columns=['Name', 'Age'])
print(df)

From Series

s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])

df = pd.DataFrame({'A': s1, 'B': s2})
print(df)

✅ From NumPy Array

```
import numpy as np

arr = np.array([[1,2], [3,4]])

df = pd.DataFrame(arr, columns=["A", "B"])
```

🔹 3. Accessing Columns in DataFrame

✅ Method 1: Using Column Name

```
print(df["Name"])
```

✅ Method 2: Dot Notation

```
print(df.Name)
```

⚠️ Works only if column name has no spaces

✅ Multiple Columns

```
print(df[["Name", "Marks"]])
```

🔹 4. Accessing Rows in DataFrame

✅ Using `loc` (Label-based)

```
print(df.loc[0])
```

👉 Access row with index label 0

✅ Using `iloc` (Position-based)

```
print(df.iloc[1])
```

👉 Access second row

✅ Slice Rows

```
print(df.iloc[0:2])
```

👉 First two rows

Assignments

Create a DataFrame for 6 students with the following columns:
Name, Age, Marks
Perform the following operations:

1. Display the complete DataFrame
2. Display only the **Name and Marks** columns
3. Access the **first and last rows** using appropriate methods
4. Display students with **Marks greater than 75**
5. Display the **Marks of the student at index 2**

Soln:

```
import pandas as pd

# Step 1: Create DataFrame
data = {
    "Name": ["Amit", "Riya", "John", "Sara", "Raj", "Neha"],
    "Age": [20, 21, 22, 20, 23, 21],
    "Marks": [85, 72, 90, 68, 76, 88]
}

df = pd.DataFrame(data)

# 1. Display complete DataFrame
print("Full DataFrame:\n", df)

# 2. Display Name and Marks columns
print("\nName and Marks:\n", df[["Name", "Marks"]])

# 3. First and last rows
print("\nFirst row:\n", df.loc[0])
print("\nLast row:\n", df.loc[len(df)-1])

# 4. Students with Marks > 75
```

print("\nMarks > 75:\n", df[df["Marks"] > 75])

 # 5. Marks of student at index 2
print("\nMarks at index 2:", df.loc[2, "Marks"])

Q2. Create a DataFrame for employees with columns:
Emp_ID, Name, Salary, Department
Perform the following operations:

1. Display the first 3 rows
2. Access the **second row using `iloc[]`**
3. Display only the **Name and Salary columns**
4. Filter employees with **Salary greater than 30,000**
5. Display the **shape and column names** of the DataFrame

Soln:

import pandas as pd

 # Step 1: Create DataFrame
data = {
"Emp_ID": [101, 102, 103, 104, 105],
"Name": ["Amit", "Riya", "John", "Sara", "Raj"],
"Salary": [25000, 32000, 40000, 28000, 35000],
"Department": ["HR", "IT", "Finance", "HR", "IT"]
}

df = pd.DataFrame(data)

 # 1. First 3 rows
print("First 3 rows:\n", df.head(3))

 # 2. Second row using iloc
print("\nSecond row:\n", df.iloc[1])

 # 3. Name and Salary columns
print("\nName and Salary:\n", df[["Name", "Salary"]])

 # 4. Salary > 30000
print("\nSalary > 30000:\n", df[df["Salary"] > 30000])

 # 5. Shape and column names
print("\nShape:", df.shape)
print("Columns:", df.columns)

🔹 Pandas Index Object

👉 Every Series/DataFrame has an **Index**

```
print(df.index)
```

✔ Output:

```
RangeIndex(start=0, stop=3, step=1)
```

🔹 What is Index?

👉 Index = **labels for rows**

✅ Custom Index

```
df = pd.DataFrame(data, index=["a", "b"])

print(df)
```

✔ Output:

```
   Name  Age
a     A   20
b     B   21
```

✅ Access using Index

```
print(df.loc["a"])
```

🔹 Key Differences

Feature	Series	DataFrame
Dimension	1D	2D
Structure	Single column	Rows + Columns
Example	`[10,20,30]`	Table

👉 **Series = Single column**
👉 **DataFrame = Table (like Excel)**
👉 `loc` = label
👉 `iloc` = position

Reindexing Series and DataFrame:

What is Reindexing?

Reindexing means **changing the index (labels)** of a Series or DataFrame.
👉 It can:

- Reorder data
- Add new labels
- Remove existing labels

Example: Reindexing Series

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

# Reindex
s_new = s.reindex(['a', 'c', 'd'])

print(s_new)
```

Output:
```
a    10.0
c    30.0
d     NaN
```

Example: Reindexing DataFrame

```
df = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
}, index=['x', 'y'])

# Reindex rows
df_new = df.reindex(['x', 'y', 'z'])
print(df_new)
```

Output:
```
     A    B
x  1.0  3.0
y  2.0  4.0
```

z NaN NaN

Filling Missing Values

df_new = df.reindex(['x','y','z'], fill_value=0)
print(df_new)

Output:
```
   A   B
x 1.0 3.0
y 2.0 4.0
z 0.   0
```

Dropping Entries

(a) Dropping from Series

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

s = s.drop('b')
print(s)
```
✔ Output:
```
a    10
c    30
```

(b) Dropping from DataFrame

👉 Drop row
```
df.drop('x', axis=0)
df
```

👉 Drop column
```
df.drop('A', axis=1)
```

Arithmetic Operations between DataFrame and Series

Operations are **aligned by index and column names**

df = pd.DataFrame({
   'A': [1, 2],
   'B': [3, 4]
}, index=['x', 'y'])

s = pd.Series([10, 20], index=['A', 'B'])

result = df + s

print(result)

DF
   A B
x 1 3
y 2 4

Series:
A   10
B   20

Output
    A   B
x 11 23
y 12 24

Function Application and Mapping

a) apply() → Apply function on DataFrame

df = pd.DataFrame({
'A': [1, 2, 3],
'B': [4, 5, 6]
})

print(df.apply(sum)) # Column-wise sum

Output:
A   6
B   15
dtype: int64

b) applymap() → Element-wise function

```
print(df.applymap(lambda x: x*2))
```

Output

   A B
0 2 8
1 4 10
2 6 12

(c) map() → For Series only

```
s = pd.Series([1, 2, 3])

print(s.map(lambda x: x*10))
```

Output:
0   10

```
1    20
2    30
dtype: int64
```

Topic	Function
Reindexing	`reindex()`
Dropping	`drop()`
Indexing	`loc[]`, `iloc[]`
Filtering	Conditions (`df[df['col'] > value]`)
Arithmetic	`+`, `-`, `*`, `/`
Apply function	`apply()`, `applymap()`, `map()`

Exercise
Q1.Create a **Pandas Series** using:

- A list
- A dictionary

Perform the following:

1. Assign custom index
2. Access elements using labels and positions
3. Perform arithmetic operation on Series
4. Display index, values, and data type

Soln:

```
import pandas as pd

# 🔷 Create Series from a list
s1 = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

# 🔷 Create Series from a dictionary
data = {'x': 100, 'y': 200, 'z': 300}
s2 = pd.Series(data)

# Display Series
print("Series from List:\n", s1)
print("\nSeries from Dictionary:\n", s2)

# 1. Custom index already assigned in s1

# 2. Access elements using labels and positions
print("\nAccess using label (s1['a']):", s1['a'])
print("Access using position (s1[0]):", s1[0])
```

```
# 3. Arithmetic operation
print("\nAfter adding 5 to Series s1:\n", s1 + 5)

# 4. Display index, values, and data type
print("\nIndex of s1:", s1.index)
print("Values of s1:", s1.values)
print("Data type of s1:", s1.dtype)
```

Q2. – Series Filtering & Handling Missing Values

Create a Series with some missing values (`NaN`).
Perform:

1. Identify missing values
2. Remove missing values
3. Replace missing values with a constant
4. Filter values greater than a given number

Sol:

```
import pandas as pd
import numpy as np

# Create Series with missing values
s = pd.Series([10, 20, np.nan, 40, np.nan, 60])

print("Original Series:\n", s)

# 1. Identify missing values
print("\nMissing Values (True = NaN):\n", s.isnull())

# 2. Remove missing values
s_no_nan = s.dropna()
print("\nSeries after removing NaN:\n", s_no_nan)

# 3. Replace missing values with a constant (e.g., 0)
s_filled = s.fillna(0)
print("\nSeries after replacing NaN with 0:\n", s_filled)

# 4. Filter values greater than a given number (e.g., > 25)
filtered = s[s > 25]
print("\nValues greater than 25:\n", filtered)
```

Q3. DataFrame Creation & Column Operations

Create a DataFrame with columns: **Name, Age, Marks**
Perform:

1. Display the DataFrame

2. Access single and multiple columns
3. Add a new column “Grade”
4. Modify existing column values

Soln:

```
import pandas as pd

# Step 1: Create DataFrame
data = {
   "Name": ["Amit", "Riya", "John"],
   "Age": [20, 21, 22],
   "Marks": [85, 90, 78]
}

df = pd.DataFrame(data)

# 1. Display the DataFrame
print("Full DataFrame:\n", df)

# 2. Access single and multiple columns
print("\nSingle Column (Name):\n", df["Name"])
print("\nMultiple Columns (Name & Marks):\n", df[["Name", "Marks"]])

# 3. Add a new column "Grade"
df["Grade"] = ["A", "A+", "B"]
print("\nAfter Adding Grade Column:\n", df)

# 4. Modify existing column values (increase Marks by 5)
df["Marks"] = df["Marks"] + 5
print("\nAfter Modifying Marks:\n", df)
```

Q4. – Row Access & Filtering

Using a DataFrame:

1. Access rows using `loc[]` and `iloc[]`
2. Display first and last rows
3. Filter rows where Marks > 80
4. Display a specific value using row and column index

Soln:

```
import pandas as pd

# Create DataFrame
data = {
   "Name": ["Amit", "Riya", "John", "Sara"],
   "Age": [20, 21, 22, 23],
   "Marks": [85, 70, 90, 78]
```

```
}

df = pd.DataFrame(data)

print("Original DataFrame:\n", df)

# 1. Access rows using loc[] and iloc[]
print("\nRow using loc[1]:\n", df.loc[1])     # Label-based
print("\nRow using iloc[2]:\n", df.iloc[2])  # Position-based

# 2. Display first and last rows
print("\nFirst Row:\n", df.head(1))
print("\nLast Row:\n", df.tail(1))

# 3. Filter rows where Marks > 80
print("\nRows with Marks > 80:\n", df[df["Marks"] > 80])

# 4. Display specific value (row 2, column 'Marks')
print("\nMarks at row index 2:", df.loc[2, "Marks"])
```

🔷 Q5. – Series vs DataFrame & Analysis

Create:

- One Series
- One DataFrame

Perform:

1. Convert Series into DataFrame
2. Apply statistical functions (`mean`, `max`, `min`)
3. Explain difference between Series and DataFrame with output
4. Display shape and structure of DataFrame

Soln:

```
import pandas as pd

# Step 1: Create a Series
s = pd.Series([10, 20, 30, 40], name="Numbers")

# Step 2: Create a DataFrame
data = {
   "Name": ["Amit", "Riya", "John", "Sara"],
   "Marks": [85, 90, 78, 88]
}
df = pd.DataFrame(data)

# Display Series and DataFrame
```

```
print("Series:\n", s)
print("\nDataFrame:\n", df)

# 1. Convert Series into DataFrame
df_from_series = s.to_frame()
print("\nSeries converted to DataFrame:\n", df_from_series)

# 2. Apply statistical functions
print("\nStatistical Functions on Series:")
print("Mean:", s.mean())
print("Max:", s.max())
print("Min:", s.min())

print("\nStatistical Functions on DataFrame (Marks column):")
print("Mean:", df["Marks"].mean())
print("Max:", df["Marks"].max())
print("Min:", df["Marks"].min())

# 3. Display shape and structure of DataFrame
print("\nShape of DataFrame:", df.shape)
print("\nStructure (info):")
print(df.info())
```
